

Digital Logic

BCA — Semester 1 — 2018 — Answer Sheet

Subject Code: CACS105

Group B

Attempt any SIX questions. [6×5=30]

11. Subtract: 675.6 – 456.4 using both 10's and 9's complement.

[5]

Answer: Given: M = 675.6, N = 456.4. We need to compute M - N.

1. Using 10's Complement:

10's complement of N (456.4) = $10^n - N$, where n = 4 (one digit more than max digits).

10's complement of 456.4 = $1000 - 456.4 = 543.6$

Now, M + 10's complement of N = $675.6 + 543.6 = 1219.2$

Since there is an end carry ($1219.2 > 1000$), discard the carry (1000):

Result = 219.2

2. Using 9's Complement:

9's complement of N (456.4) = $(10^n - 10^{-m}) - N$, where n=3 integer digits, m=1 decimal place.

$999.9 - 456.4 = 543.5$

Now, M + 9's complement of N = $675.6 + 543.5 = 1219.1$

Since there is an end carry, add it back:

Result = $219.1 + 0.1 = 219.2$

Verification: $675.6 - 456.4 = 219.2$ ✓

12. What is universal logic gate? Realize NAND and NOR as universal logic gates.

[1+2+2]

Answer: Universal Logic Gate:

A universal logic gate is a gate that can be used to implement any Boolean function without requiring any other type of gate. By combining only universal gates, we can construct AND, OR, and NOT operations, and therefore any digital circuit. The two universal gates are **NAND** and **NOR**.

1. NAND as Universal Gate:

NOT using NAND: Connect both inputs together: $Y = (A \cdot A)' = A'$

AND using NAND: NAND followed by NOT: $Y = ((A \cdot B)')' = A \cdot B$

OR using NAND: Using De Morgan's theorem: $Y = (A' \cdot B')' = A + B$

2. NOR as Universal Gate:

NOT using NOR: Connect both inputs together: $Y = (A + A)' = A'$

OR using NOR: NOR followed by NOT: $Y = ((A + B)')' = A + B$

AND using NOR: Using De Morgan's theorem: $Y = (A' + B')' = A \cdot B$

Proof of NAND as Universal:

Any Boolean function can be expressed in Sum of Products (SOP) form, which uses AND and NOT operations. Since NAND can implement both AND and NOT, it is universal.

Proof of NOR as Universal:

Any Boolean function can be expressed in Product of Sums (POS) form, which uses OR and NOT operations. Since NOR can implement both OR and NOT, it is universal.

13. Simplify (using K-map) the given Boolean function F in both SOP and POS using don't care conditions D.

[2+3]

Answer: Note: The original question contains corrupted symbols for the Boolean function. Based on typical exam patterns, the function appears to be:

$$F(A,B,C,D) = \Sigma(1,3,5,7,8,10,12,14) \text{ with don't cares } d(A,B,C,D) = \Sigma(0,2,4,6,9,11)$$

K-map for SOP (grouping 1s):

CD 00 01 11 10 AB 00 d 1 1 d 01 d 1 1 d 10 1 d d 1 11 1 d d 1

Groupings:

- Group 1: All cells in rows AB=00 and AB=01 where C=1 (columns 01, 11) → B=0, C=1, D varies → term: A'C
- Group 2: All cells in rows AB=10 and AB=11 where D=0 (columns 00, 10) → A=1, D=0, B varies → term: AD'

Simplified SOP: $F = A'C + AD'$

K-map for POS (grouping 0s):

The 0s are at positions: 13, 15 (with don't cares at 0,2,4,6,9,11)

CD 00 01 11 10 AB 00 d 1 1 d 01 d 1 1 d 10 1 d d 1 11 1 d 0 1

The 0s are at (11,11)=15 and (11,01)=13, with adjacent don't cares.

Group: A=1, B=1, C=1, D varies → maxterm: (A' + B' + C')

Wait — with the given pattern, a better grouping using don't cares for POS:

Group all 0s with don't cares to cover the complement.

Simplified POS: $F = (A + C)(A' + D')$

Verification: $(A + C)(A' + D') = AA' + AD' + A'C + CD' = 0 + AD' + A'C + CD' = AD' + A'C$ (since CD' is covered by the other terms)

14. Define encoder. Draw logic diagram and truth table of octal-to-binary encoder.

[1+4]

Answer: Encoder:

An encoder is a combinational circuit that converts information from one format or code to another, typically converting 2^n input lines into n output lines. It performs the reverse operation of a decoder. A priority encoder assigns priority to inputs so that if multiple inputs are active, the highest priority input is encoded.

Octal-to-Binary Encoder (8-to-3):

Converts 8 octal inputs (D_7 to D_0) into 3 binary outputs (A, B, C).

Truth Table:

Active Input	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A	B	C
00000000	1	0	0	0	0	0	0	0	0	0	0
10000001	0	1	0	0	0	0	0	0	0	1	0
20000010	0	0	1	0	0	0	0	0	1	0	0
30000100	0	0	0	1	0	0	0	0	1	1	0
40001000	0	0	0	0	1	0	0	0	0	0	1
50010000	0	0	0	0	0	1	0	0	0	1	1
60100000	0	0	0	0	0	0	1	0	1	0	1
71000000	0	0	0	0	0	0	0	1	1	1	1

Boolean Expressions:

$$A = D_7 + D_6 + D_5 + D_4$$

$$B = D_7 + D_6 + D_3 + D_2$$

$$C = D_7 + D_5 + D_4 + D_0$$

Logic Diagram:

Three OR gates:

- OR gate 1 (output A): inputs D_7, D_6, D_5, D_4
- OR gate 2 (output B): inputs D_7, D_6, D_3, D_2
- OR gate 3 (output C): inputs D_7, D_5, D_4, D_0

Each D input connects to the appropriate OR gates as specified above.

15. What is D flip-flop? Explain clocked RS flip-flop with its logic diagram and truth table.

[1+4]

Answer: D Flip-Flop (Data/Delay Flip-Flop):

A D flip-flop is a type of flip-flop that has a single data input (D) and a clock input. It captures the value of the D input at a specific edge of the clock signal (rising or falling) and holds that value at the output Q until the next clock edge. It eliminates the indeterminate state found in RS flip-flops. The output Q follows the input D after the clock edge: $Q(t+1) = D$.

Clocked RS Flip-Flop:

A clocked RS flip-flop is a sequential circuit with two inputs (S = Set, R = Reset) and a clock input (CLK). The outputs change only when the clock signal is active (typically on the rising edge).

Logic Diagram:

A clocked RS flip-flop consists of two NAND gates cross-coupled to form the basic latch, with two additional NAND gates (gating circuits) for the S and R inputs. The clock signal is connected to both gating NAND gates.

$S \text{ ----[NAND]----+---- } Q \text{ || CLK --+----[NAND] || } R \text{ ----[NAND]----+---- } Q' \text{ || CLK --+----[NAND] |}$

Truth Table (Characteristic Table):

S	R	CLK	$Q(t+1)$	State
0	0	↑	$Q(t)$	No change
0	1	↑	0	Reset
1	0	↑	1	Set
1	1	↑	?	Invalid/Indeterminate

Drawbacks of Clocked RS Flip-Flop:

1. Invalid state when $S=R=1$ (both outputs may become 0, violating $Q = Q'$)
2. The forbidden input combination must be avoided
3. No toggle capability (cannot change state with single input)

16. Design MOD-5 counter with state and timing diagram.

[2+1+2]

Answer: MOD-5 Counter:

A MOD-5 counter has 5 states (0 to 4) and requires 3 flip-flops (since $2^2 = 4 < 5 \leq 2^3 = 8$). It counts from 000 to 100 and then resets to 000.

State Diagram:

000(0) $\rightarrow$ 001(1) $\rightarrow$ 010(2) $\rightarrow$ 011(3) $\rightarrow$ 100(4) $\rightarrow$ (000)

State Table (using T flip-flops):

Present State	$Q_2Q_1Q_0$	Next State	$Q_2Q_1Q_0$	T_2	T_1	T_0
000001001	000	001	001	0	0	1
001010011	001	010	010	1	0	1
010011001	010	011	011	0	1	1
011100111	011	100	100	1	1	0
100000100	100	000	000	1	0	0

K-map Simplification:

$$T_2 = Q_2 \cdot Q_1 + Q_2 \cdot Q_0'$$

$$T_1 = Q_1$$

$$T_0 = Q_0' + Q_0 \cdot Q_2' \text{ (simplified: } T_0 = 1 \text{ for states 0-3, 0 for state 4)}$$

Actually, simpler with T flip-flop excitation ($T = Q \oplus Q_{\text{next}}$):

$$T_2 = Q_2'Q_1Q_0 + Q_2Q_1'Q_0' = Q_2Q_0 + Q_2'Q_1' \text{ (after simplification)}$$

$$T_1 = Q_1 \oplus Q_1 = 0$$

$$T_0 = Q_0'Q_2' + Q_0Q_2 = Q_2 \oplus Q_0' = Q_2'Q_0' + Q_2Q_0$$

Wait — let me recalculate directly:

From state table: $T_2 = 1$ when (011 $\rightarrow$ 100) and (100 $\rightarrow$ 000), so $T_2 = Q_2Q_1' + Q_2Q_0$

$T_1 = 1$ when (001 $\rightarrow$ 010) and (011 $\rightarrow$ 100), so $T_1 = Q_1 \oplus Q_0$ (approximately)

$T_0 = 1$ when states change in LSB, which is every clock except 100 $\rightarrow$ 000

So $T_0 = Q_0'$ (since when $Q_0=1$, $T_0=0$)

Timing Diagram:

CLK: 
Q₂: 
Q₁: 
Q₀: 

States: 000 $\rightarrow$ 001 $\rightarrow$ 010 $\rightarrow$ 011 $\rightarrow$ 100 $\rightarrow$ 000 $\rightarrow$ 001...

After 5 clock pulses, the counter returns to 000.

17. Design a 4-bit serial into parallel-out shift register with timing diagram.

[3+2]

Answer: 4-bit Serial-In Parallel-Out (SIPO) Shift Register:

A SIPO shift register accepts data serially (one bit at a time) through a single input line and outputs all bits in parallel after all bits have been shifted in.

Design:

Four D flip-flops connected in cascade:

- Serial Input (SI) $\rightarrow D_4 \rightarrow Q_4 \rightarrow D_3 \rightarrow Q_3 \rightarrow D_2 \rightarrow Q_2 \rightarrow D_1 \rightarrow Q_1$
- Common clock (CLK) connected to all flip-flops
- Parallel outputs: Q_4, Q_3, Q_2, Q_1
- Clear input (optional) to reset all flip-flops to 0

Operation:

With each clock pulse, the data shifts one position to the right:

- $Q_4(t+1) = Q_4(t)$
- $Q_3(t+1) = Q_4(t)$
- $Q_2(t+1) = Q_3(t)$
- $Q_1(t+1) = SI(t)$

Timing Diagram (shifting in data 1011):

CLK: $\uparrow \uparrow \uparrow \uparrow$ SI: 1 0 1 1 Q_4 : 0 $\rightarrow$ 1 1 $\rightarrow$ 0 0 $\rightarrow$ 1 1 $\rightarrow$ 1 Q_3 : 0 $\rightarrow$ 0 0 $\rightarrow$ 1 1 $\rightarrow$ 0 0 $\rightarrow$ 1 Q_2 : 0 $\rightarrow$ 0 0 $\rightarrow$ 0 0 $\rightarrow$ 1 1 $\rightarrow$ 0 Q_1 : 0 $\rightarrow$ 0 0 $\rightarrow$ 0 0 $\rightarrow$ 0 0 $\rightarrow$ 1

After 4 clock pulses, parallel output $Q_4Q_3Q_2Q_1 = 1011$

Applications:

- Serial-to-parallel data conversion
- Digital delay lines
- Data buffering in communication systems
- Keyboard encoding

Group C

Attempt any TWO questions. [2 $\times$ 10=20]

18. Write difference between PLA and PAL. Design a PLA circuit with given functions. $F(A,B,C) = \Sigma(2,3,5)$, $F(A,B,C) = \Sigma(0,4,5,7)$. Design PLA program table also.

[3+7]

Answer: Difference between PLA and PAL:

Aspect	PLA	(Programmable Logic Array)	PAL	(Programmable Array Logic)	AND
Array	Programmable	OR Array	Programmable	Fixed	Flexibility
Cost	Higher	Lower	Speed	Slower (more fusible links)	Faster
Product terms	Can share product terms across outputs	Each output has dedicated product terms			

PLA Design:

$$F(A,B,C) = \Sigma(2,3,5) = A'BC' + A'BC + AB'C$$

$$F(A,B,C) = \Sigma(0,4,5,7) = A'B'C' + AB'C' + AB'C + ABC$$

Simplifying:

$$F = A'B(C' + C) + AB'C = A'B + AB'C$$

$$F = B'C'(A + A) + AB'C + ABC = B'C' + AB'C + ABC$$

Product terms:

$$P_1 = A'B \text{ (for } F_1)$$

$$P_2 = AB'C \text{ (for } F_1, F_2)$$

$$P_3 = B'C' \text{ (for } F_2)$$

$$P_4 = ABC \text{ (for } F_2)$$

PLA Program Table:

Product Term	ABCF	F_1	F_2	A'B	01-10	AB'C	10111	B'C'	-0001	ABC	11101
--------------	------	-------	-------	-----	-------	------	-------	------	-------	-----	-------

(1 = true, 0 = complemented, - = don't care)

Circuit Diagram:

Inputs A, B, C go through inverters to provide A, A', B, B', C, C' to the AND array.

- AND gate 1: $A' \cdot B$ (connects A' and B)
- AND gate 2: $A \cdot B' \cdot C$ (connects A, B', C)
- AND gate 3: $B' \cdot C'$ (connects B', C')
- AND gate 4: $A \cdot B \cdot C$ (connects A, B, C)

OR array:

- $F_1 = \text{OR}(P_1, P_2)$
- $F_2 = \text{OR}(P_2, P_3, P_4)$

19. Define D flip-flop. Design a Master-slave flip-flop by using JK flip-flop along with its circuit diagram and truth table.

[2+8]

Answer: D Flip-Flop:

A D flip-flop (Data or Delay flip-flop) is a sequential logic device with a single data input (D) and a clock input. It captures the input value at the active clock edge and holds it at the output Q. The next state equation is $Q(t+1) = D$. It eliminates the invalid state problem of RS flip-flops since D can only be 0 or 1, never ambiguous.

Master-Slave JK Flip-Flop:

A master-slave flip-flop uses two cascaded flip-flops (master and slave) triggered by opposite clock phases to eliminate race conditions. Data is latched into the master on the clock's rising edge and transferred to the slave on the falling edge.

Design using JK Flip-Flops:

The master-slave JK flip-flop consists of two JK flip-flops:

- **Master FF:** Enabled when CLK = 1 (high). Accepts J and K inputs.
- **Slave FF:** Enabled when CLK = 0 (low). Accepts outputs from the Master.
- **Inverter:** CLK is inverted for the Slave FF.

Operation:

1. When CLK = 1: Master is active, Slave is inactive (holds previous output).
2. When CLK transitions to 0: Master becomes inactive (latches data), Slave becomes active and receives Master's output.
3. The output Q only changes after the clock falling edge, preventing race-around condition.

Truth Table:

JKCLKQ(t+1)State 00↓Q(t)No change 01↓0Reset 10↓1Set 11↓Q'(t)Toggle

Circuit Diagram:

J ----+----[Master JK FF]----+---- Q | (CLK = direct) | K ----+ | | CLK --+----[INVERTER] | | | +----[Slave JK FF]
| (CLK = inverted) +----- J_slave = Q_master K_slave = Q'_master

Advantages of Master-Slave Design:

- Eliminates race-around condition (unlike edge-triggered designs)
- Output changes only once per clock cycle
- More reliable for sequential circuit design

20. Write down the difference between asynchronous and synchronous counter. Design a 4-bit binary ripple counter along with its circuit, state and timing diagram.

[3+7]

Answer: Difference between Asynchronous and Synchronous Counters:

Aspect	Asynchronous (Ripple) Counter	Synchronous Counter
Clock	Only first flip-flop receives external clock; others are triggered by previous stage	All flip-flops receive the same external clock simultaneously
Speed	Slower due to cumulative propagation delay	Faster; all stages change simultaneously
Design	Simpler design and wiring	More complex design with additional logic gates
Decoding errors	Possible due to ripple delay (transient states)	No decoding errors; all outputs change together
Applications	Simple counting, frequency division, low-speed applications	High-speed counting, precise timing applications
Examples	MOD-8 ripple counter, MOD-16 ripple counter, Ring counter, Johnson counter	synchronous decade counter

4-bit Binary Ripple Counter:

A 4-bit ripple counter uses 4 JK flip-flops in toggle mode ($J=K=1$). Each flip-flop divides the frequency by 2.

Circuit Diagram:

CLK --+>[JK FF0]--Q■--+>[JK FF1]--Q■--+>[JK FF2]--Q■--+>[JK FF3]--Q■ | $J=K=1$ | $J=K=1$ | $J=K=1$ | $J=K=1$ |
 J=K=1 | | | +----(falling) +----(falling) +----(falling) +----(falling)

- External CLK connects to FF0 clock input
- Q■ connects to FF1 clock input (negative edge triggered)
- Q■ connects to FF2 clock input
- Q■ connects to FF3 clock input

State Diagram:

0000 → 0001 → 0010 → 0011 → 0100 → 0101 → 0110 → 0111 → 1000 → 1001 → 1010 → 1011 → 1100 → 1101 → 1110 → 1111 → (0000)

Counts from 0 to 15 (16 states), then resets.

Timing Diagram:



- Q■ toggles every clock cycle (frequency = $f/2$)
- Q■ toggles every 2 clock cycles (frequency = $f/4$)
- Q■ toggles every 4 clock cycles (frequency = $f/8$)
- Q■ toggles every 8 clock cycles (frequency = $f/16$)

The counter completes one full cycle every 16 clock pulses.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.